

BusSim: Modeling the GT Stinger Shuttle

Team 2

Anthony Jungwoo Ahn, Anushka Chaudhari, Grace Ann Driskill, Kyle
Joseph Peters, Neelkanth Shriniwas Ganediwal, Zi'an Dong

GitHub repo: <https://github.gatech.edu/aahn33/mod-sim-project/tree/main>

Abstract

The GT Bus Route Simulator is an intricate simulation tool built to assess traffic flows and public transportation efficacy. Our project adopts a time-based methodology to mimic the dynamic operations of buses and commuters navigating a network of bus stops, traffic signals, and roadways. Our project leverages a timestep-driven approach to simulate the movement of buses and passengers across a bus route consisting of bus stops, traffic lights, and road segments. Buses can exhibit several behaviors, or events, in the simulation such as arriving at a bus stop, departing from a bus stop, traveling across a road segment, or stopping at a traffic light. The model was validated using real-world data from the GT Bus API, and then used to analyze the effects of bus starting points on passenger wait times.

Project Description

Buses are an essential form of transit on college campuses. Given the scarcity and high cost of parking spots, as well as the potential traffic jams caused by too many vehicles, communal transit emerges as a more efficient alternative to walking for students needing to cover broader areas of the campus swiftly when cycling isn't an option. Therefore, for students with extremely busy schedules, it is important for the buses to operate in a timely, efficient fashion. Our project plans to develop a simulation model to model the behavior of bus transportation systems.

Specifically, we seek to model the behavior of the GT bus systems. We will collect location and capacity information for the GT buses to test and validate our model. We plan to first develop our model for the GT bus “blue route”, which wraps around Georgia Tech’s campus, between the hours of 7 AM and 7 PM. Depending on how long this takes, we might try to model other routes as well. Once we have refined our model, we will perform multiple experiments to see the effect of changing different model parameters on different model results, such as average wait time, bus bunching, and more. We hope to also develop visual simulations for such experiments.

Literature Review

Our project plans to develop a simulation model for bus systems. One use of modeling for bus systems is to predict bus arrival and departure times. Automatic vehicle locating (AVL), Automatic Passenger Count (APC), automatic fare collection (AFC), global positioning system (GPS), historical data allows for the creation of models that can be useful for passengers using buses or researchers trying to optimize buses. One common approach is to use machine learning methods such as artificial neural networks (ANN) or support vector machines (SVM) [1] [2] [3]. Another approach is to use Kalman filter models [4][5], which is a state estimation technique that is useful when working with noisy data [6]. Finally, hybrid models can also be used for bus arrival predictions; these combine Kalman filters and SVMs [7][8], or Kalman filters and statistical estimations [9][10] or other combinations of multiple types of methods.

There is a significant body of research within transit system studies which focuses on modeling behaviors and dynamics at bus stops and along routes. Rather than directly forecasting timings, it focuses rather on understanding and optimizing specific components of bus transit systems. These components range from bus queue times at stops to the simulation of entire bus terminals, which can offer insights into the system and its potential areas for improvements. An innovative approach to leveraging Automatic Vehicle Location (AVL) and Automatic Passenger Count (APC) data was created by Shalaby and Farhan [11] which provided a comprehensive framework for predicting an accurate arrival and departure time. It demonstrated how integrating real time and historical data can enhance reliability by introducing the Kalman filter-based model, that accounts for the dynamic nature of urban transit. Furthermore, Comi and Polimeni proposed a time-based series approach for forecasting time travel and data from Automated Vehicle Monitoring (AVM) of bus lines sharing the road lanes in Rome and Italy. They considered travel time patterns and city structure, emphasizing their importance. Moreover, Mete, Celik, and Gul, introduces time series models and explores the application of statistical techniques to model and forecast the bus arrival times, which again reinforced the effectiveness of time series analysis in capturing patterns and trends in transit data. Different models predicted the bus arrival time for the route and this one received the most accurate results.

Recent studies on bus transit systems extend beyond simple arrival and departure time predictions, focusing on comprehensive system optimizations and addressing specific operational challenges such as bus bunching. These research efforts utilize a range of models

and simulations to propose strategic improvements that enhance overall system efficiency and reliability without compromising operational constraints.

Optimizing Wait Times: A study of the CMU bus system focused on minimizing wait times while considering labor costs [14]. The model analyzed bus arrival times and passenger counts to recommend the optimal number of buses for different times of day, potentially reducing wait times significantly.

Addressing Bus Bunching: Another study proposed a model to optimize bus frequencies, addressing the issue of bus bunching [15]. The model balances the need to reduce bunching (which can cause irregular service) with the importance of minimizing passenger wait times.

Multi-Strategy Evaluation: Research on Kuala Lumpur's Route U32 investigated the impact of several strategies to improve bus service reliability [16]. A simulation model compared the effectiveness of headway-based departure and previous headway holding strategies, demonstrating that a combination of these approaches offered the best results.

These examples underscore the value of optimization and simulation in improving bus transit efficiency, reliability, and overall passenger experience.

Du-Bus is a real-time bus waiting time estimation system that leverages multi-source data to approximate the real-time locations of buses without GPS sensors. It integrates data from various sources such as GPS-equipped buses, transportation networks, traffic conditions, user mobility, and uses this to model bus timetables and real-time travel times. This system employs a bidirectional LSTM model to estimate bus departure intervals and a DNN to calculate travel times, ultimately estimating waiting times for any city station [17]. Du-Bus demonstrated superior accuracy over traditional methods and has been implemented in over 20 major cities in China, showcasing its practical effectiveness. This system can be used in a data-based model for our bus system by integrating actual data to predict bus arrival and departure times from diverse sources. The ways of estimating bus departure intervals and travel times can be applied to develop predictive models in intelligent transportation systems, improving the accuracy of both arrival and departure predictions. Another paper I looked at uses ML/DL algorithms for real-time prediction of bus arrival times including Optimal Least Square (OLS) linear regression, Support Vector Regression (SVR), and Fully Connected Neural Networks (FNN) [18]. It focuses

on improving public transportation systems by predicting bus arrival times at various stops, considering real-time traffic conditions. This can be applied to data-based models to enhance accuracy of predictions and facilitate more efficient and reliable public transportation services [19].

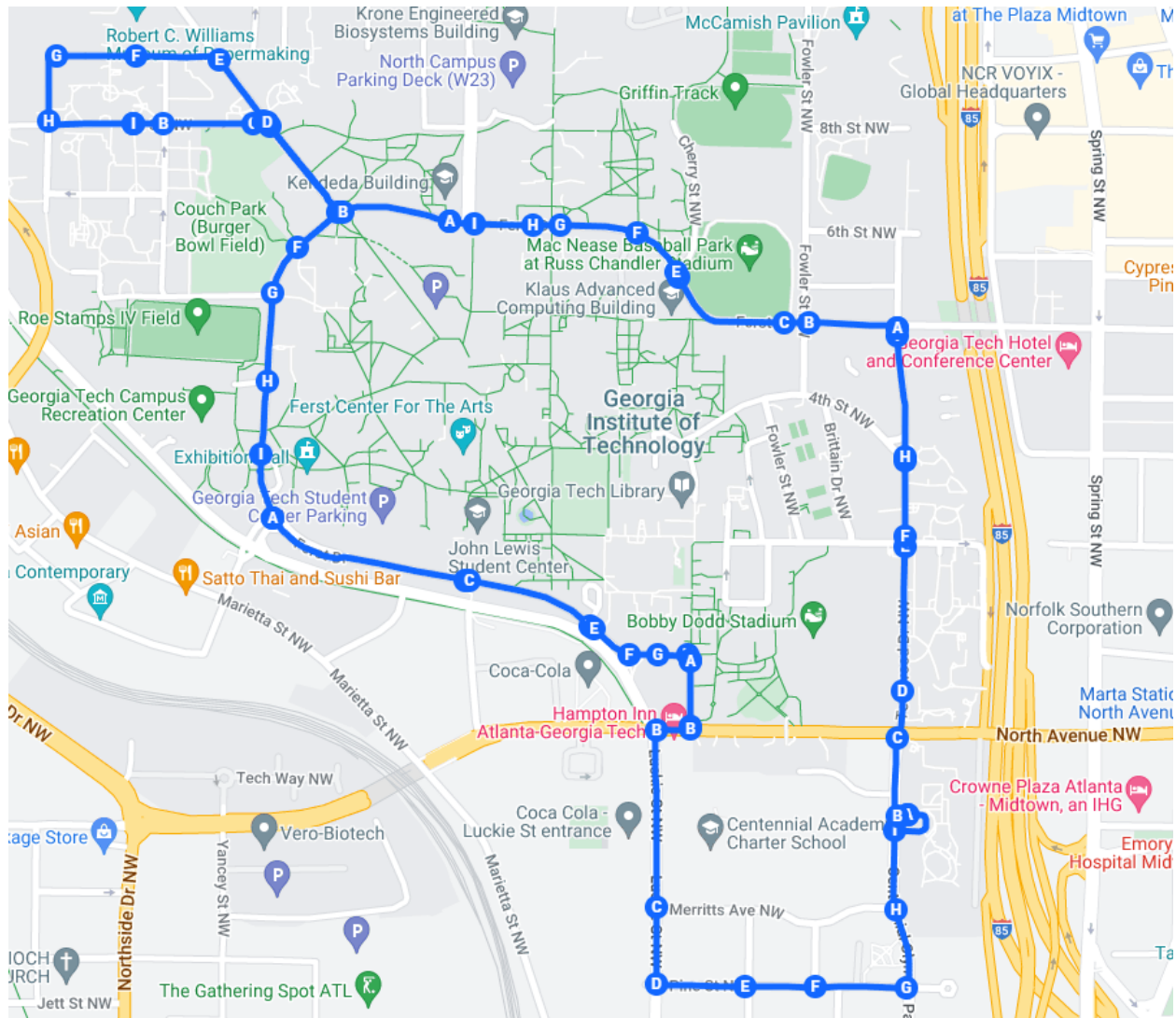
We hope to build upon this existing literature and iteratively develop a simulation model to not only predict bus arrival times, but also produce optimizations to reduce negative effects like bus bunching. We will be validating and contextualizing our model using real data from Georgia Tech's bus system.

Conceptual Model

The conceptual model serves as the basis for our simulation. For simplicity, our conceptual model simulates a single bus route. We used the conceptual model to analyze the effects of varying bus schedules and configurations on passenger wait times. The model leverages a time-driven approach to track the movement of buses and passengers across a network of bus stops, traffic signals, stop signs, etc.

Bus Route Creation:

We decided to choose the blue bus route from the Georgia Institute of Technology's bus system to simulate. We used Google maps to trace out a driving route along the blue bus route which contained stops at every turn, stop sign, traffic light, cross walk, and bus stop. After generating the route on Google Maps, we were able to export the route into a kml file which is a file type that is used to store geospatial data. Google maps didn't support adding types to each stop, so we had to create a list of labels for each type of stop on the map.



To make our simulation/experiments easier, we decided to convert this 2D bus route into a 1D representation of the route. If we used a 2D representation, moving the bus in the right x, y coordinates as we stepped through the simulation would be difficult. The 1d representation, though, would make it easy to step the bus along the route as we would only need to calculate how far right or left to move the bus without worrying about a second dimension.

We can create a 1D representation of the bus system by calculating the distance along the route between consecutive stops. We would then put the first stop at $X = 0$ and put each subsequent stop (i) at $X_i + \text{distance between stop } i \text{ and } i-1$. In our simulation, we would then have to calculate how far the bus travels and move the bus that distance to the right. We would have to handle the edge case at the end of the route by moving the bus back to $X=0$.

Core Logics:

```
route = Route()
route.add_traffic_entities # bus stops, stop signs, etc.
total_waiting_time = 0 # benchmark to be optimized

While time in valid_period:
    if time == scheduled_bus_departure_time:
        route.append(bus)

    for bus in bus_list:
        bus.update() # this include update position (if driving) or picking up passengers, etc.
        if bus in terminal_station:
            route.remove(bus)

    for stop in stop_list:
        stop.update() # update number of passengers at each stop
        total_waiting_time += timestep * stop.passengers

    for traffic_light in traffic_lights:
        traffic_light.update # update traffic light status (green or red)

    time += timestep
```

Bus Update Logic:

Let $B(t)$ denote the status of Bus, $S(t)$ denote the position of bus at time t , $v(t)$ denote the velocity, $\theta_{(t)}$ denote the congestion index on at time t , and λ_1 denote the bus constant acceleration, λ_2 denote the bus constant deceleration.

- If $B(t) ==$ waiting for traffic light:
 - If traffic light == green:
 $B(t) =$ accelerate
 - Else:
continue
- If $B(t) ==$ stop at stop sign:
 $B(t) =$ accelerate
- If $B(t) ==$ picking up:
 - If (bus.passengers $\geq$ max_capacity or bus.stop.passenger_amount == 0):
 $B(t) =$ accelerate
 - Else
suppose bus picks up 1 passengers in each time step
bus.passengers += min(1, bus.stop.passengers)
- If $B(t) ==$ dropping off:
 - # bus.drop_off(stop) is how many passengers the bus needs to drop off at this stop

If (bus.drop_off(stop) == 0:

$B(t)$ = picking up

Else:

suppose bus drops off 1 passengers in each time step

bus.passengers 0= min(1, bus.drop_off(stop))

- If $B(t)$ == accelerate:

$$v_{updated} = \min(v(t) + \lambda_1 \times timestep, \theta_i(t) \times bus.road.speed)$$

$$s(t) \approx s(t) + \frac{v(t) + v_{updated}}{2} \times timestep$$

$$v(t) = v_{updated}$$

If $v(t)$ == $\theta_i(t) \times bus.road.speed$:

$B(t)$ = constant speed

- If $B(t)$ == constant speed:

$$s(t) = s(t) + v(t) \times timestep$$

If min(next traffic light, next stop sign, next bus stop) - $s(t) \leq c$:

$B(t)$ = decelerate

- If $B(t)$ == decelerate:

$$v_{updated} = \max(v(t) - \lambda_2 \times timestep, 0)$$

$$s(t) \approx s(t) + \frac{v(t) + v_{updated}}{2} \times timestep$$

$$v(t) = v_{updated}$$

If $v(t) = 0$:

If $s(t)$ == next bus stop:

$B(t)$ = dropping off

If $s(t)$ == next stop sign:

$B(t)$ = stop at stop sign

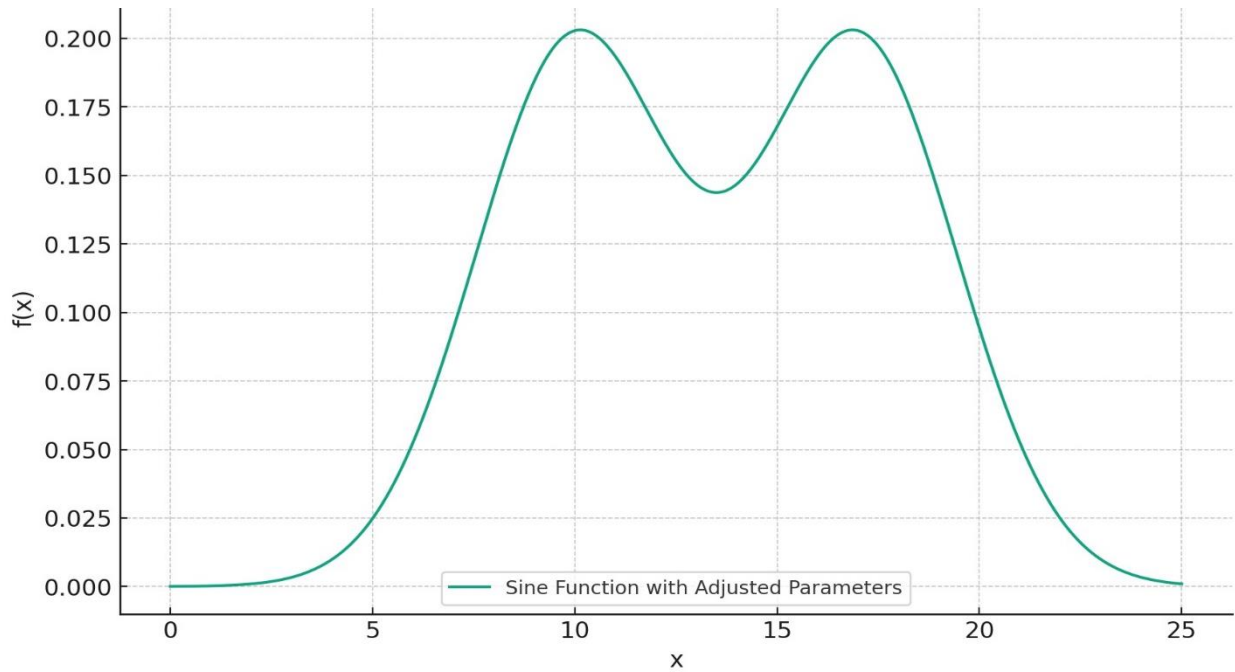
If $s(t)$ == next traffic light:

$B(t)$ = waiting for traffic light

Note:

The regression formula for $\theta_{(t)}$ is the following:

$$\theta(t) = \frac{1}{2\sqrt{2\pi}} \left(\exp\left(-\frac{(t-10)^2}{12}\right) + \exp\left(-\frac{(t-17)^2}{12}\right) \right) \times \frac{1}{0.0943}$$



Plot of $\theta_{(t)}$ without scaling

Stop Update Logic:

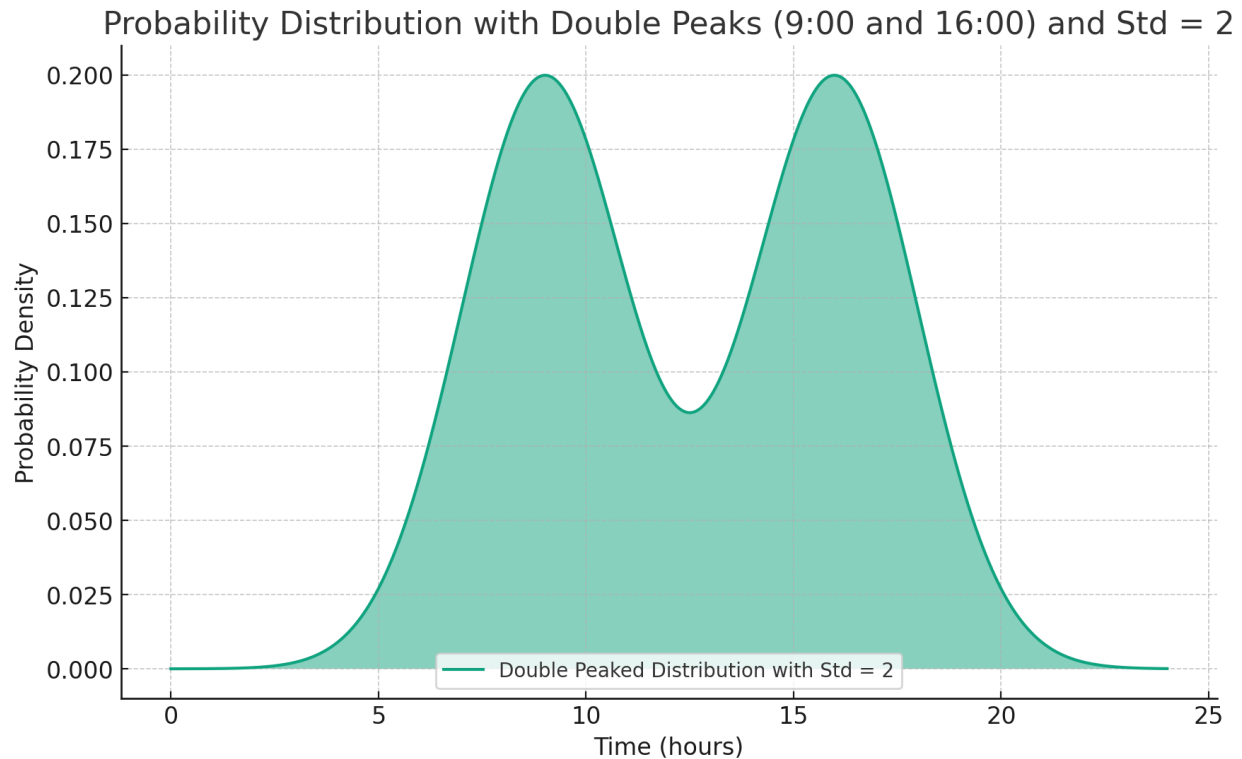
Let $p(t)$ be the rate of new passengers arriving at a bus stop at time t .. A double peak normal distribution centered around 9am and 4pm respectively will be used for $p(t)$.

At each time step, the number of passengers at a stop is updated:

$$stop.passengers += rand(0, 2) \cdot p(t) \times timestep + \epsilon$$

$$p(t) = \frac{\alpha}{2\sqrt{2\pi}} \left(e^{-\frac{(x-9)^2}{8}} + e^{-\frac{(x-16)^2}{8}} \right)$$

Where α is the scaling factor at each stop to determine its business.



Key Abstractions:

- **Buses:** Represented as objects with attributes like status (accelerating, waiting, etc.), position, velocity, passenger capacity, and a function to update their state based on events.
- **Bus Stops:** Represented as objects with attributes like location, current number of waiting passengers, and a function to update the number of passengers based on arrival rates.
- **Traffic Signals/ Stop Signs/ Crosswalks:** Represented as objects which will affect vehicle movement during the "update" function.
- **Passengers:** Represented as objects. Initialized with pick-up bus stops and drop-off bus stops
- **Route:** Represented as an object which integrates all the traffic entities (buses, bus stops, stop signs, crosswalks, etc.)

Core Events:

- Bus arrival at a scheduled stop.
- Bus departure from a stop after picking up/dropping off passengers.
- Bus movement along a road segment, considering factors like speed limits and congestion (which could be varied across different times of the day).
- Bus encountering a traffic entity (e.g. traffic lights, stops, etc) and potentially stopping/starting.

Assumptions and Limitations:

- The model assumes a fixed and deterministic road network with known speed limits.
- The model assumes the congestion index distribution follow the same time distribution pattern for all road segments (since GT bus route is a relatively small and condensed route)
- Bus movements on a road segment only contains acceleration, deceleration, and cruising. A more sophisticated model could consider lane changes and dynamic traffic patterns.
- Passenger arrival rates to bus stops follow a simplified normal distribution. Real-world arrival patterns might be more complex and require additional data sources.
- Bus boarding/onboarding times are not explicitly modeled. The model assumes a fixed number of passengers added/removed per timestep for simplicity.

This conceptual model captures the essential dynamics of a bus route, allowing us to analyze how factors like bus different bus frequencies and departure schedules can affect passenger wait times. By simulating different scenarios, we can gain valuable insights to optimize bus operations and improve public transportation efficiency.

Simulation

In the simulation, there exist several entities, which hold attributes, and a simulation driver which helps create and drive the simulation. The core logic for the bus behavior acknowledges various states (waiting, accelerating, cruising at constant speed, and decelerating) transitioning smoothly between these states to mimic realistic bus driving patterns.

Bus Behavior Model: In the simulation, buses transitioned through states including waiting, accelerating, cruising, and decelerating to reflect realistic traffic scenarios. Buses react to traffic lights, execute pick-ups and drop-offs at stops, and perform driver breaks, with each event carefully updating the bus's status within the simulation.

Passenger Wait Time Calculation: Passenger wait times are calculated by accumulating the time passengers spend at stops, with the simulation adjusting the number of passengers in response to the simulated time of day and random variations to emulate real-world unpredictability.

Bus Stop Passenger Dynamics: Bus stops are dynamically updated following a normal distribution of arrival times, designed to match typical passenger flow and incorporating

elements to simulate randomness in passenger arrivals. Each new passenger is individually tracked, adding complexity and realism to the simulation.

Data-Driven Parameterization: The simulation uses the GT Bus API data to parameterize elements like bus acceleration, velocity, and positions, ensuring that the behavior of buses closely mimics actual traffic conditions, providing a robust platform for analyzing and improving bus route operations.

Deceleration - a bus may need to decelerate upon approaching a bus stop or traffic light, or it may resume acceleration once it has finished boarding passengers, with the goal of maintaining an optimal flow within the constraints of traffic conditions and passenger needs.

Bus (bus.py)

- A bus is identified by the bus id.
- It contains attributes including the capacity, position of the bus, velocity, an array of passengers, a value to determine if it can pass a traffic light, and the wait time for a stop sign and crosswalk.
- It also contains a status attribute which shows the status of the bus such as accelerating, decelerating, cruising, stopped, etc.
- There includes a method to wait for the traffic light. When a bus comes across a traffic light, the method checks the state of the light. If the light is green, the bus's status is set to accelerating so it can begin moving. If the light is red, the bus remains in the waiting for traffic light status.
- A method to stop at the stop sign is also implemented. This method handles the bus's behavior when it stops at a stop sign and decrements the stop sign wait time based on the time step. When the wait time reaches zero, the bus can proceed, and the status is set to accelerating.
- A similar method to wait for the crosswalk is implemented, which manages the wait time at the crosswalks. It decrements the wait time for the cross walk until the bus can accelerate again.
- A pick up passengers method is implemented where a bus is at a bus stop and it will pick passengers up. It checks if there is space on the bus and if there are passengers waiting. It fills up the available seats till capacity and adjusts the status.
- Similarly, a drop off passengers method is implemented. When the bus is at a bus stop, the method checks for passengers whose destination is the current stop and removes

them from the bus. If passengers are dropped off, it continues picking up passengers. Else, it picks up new passengers.

- A method is implemented to handle the bus's acceleration. It increases the bus's speed based on its acceleration attribute up to the speed limit while also taking the congestion index of the route into account. It updates the bus's position and based on traffic objects ahead, may change the status to "cruising" or another.
- Another method is implemented to handle the deceleration as well. The bus's velocity is reduced based on the deceleration rate and the position is updated to reflect the deceleration. If the bus comes to a complete stop, it checks the type of traffic object ahead and accordingly adjusts the status.
- The cruise method maintains the bus's velocity and updates the position accordingly. It checks for upcoming traffic objects and if it needs to slow down, it accordingly changes the status to decelerating.
- The method, get breaking distance, is implemented which calculates the distance needed for the bus to stop based on its current velocity and deceleration rate. It is used for determining when the bus should start slowing down as it approaches traffic objects.
- A method, get location, is implemented which retrieves the traffic object that the bus is currently at. This could include a stop sign, traffic light, bus stop, etc.
- A method, get next location, is implemented to retrieve the next traffic object the bus will encounter based on its current trajectory and position on the route.

Bus Stop (busStop.py)

- The bus stop is identified by its stop id.
- It contains a queue of passengers and maintains attributes including passenger count, position of the bus stop along the route, the route the bus stop is on, and the scaling factor for passenger generation.
- It includes methods to add, remove, and get passengers and the count.
- An update method is implemented which updates the attributes for the bus stop with new passengers.

Cross Walk (crossWalk.py)

- The crosswalk contains attributes position and is identified by an id.
- Crosswalks are generated and placed on the route
- When buses come across a walk time, the bus waits for a certain amount of time defined in the bus class

Passenger (passenger.py)

- The passenger is identified by a passenger id.
- It contains attributes including the stop where the passenger is currently waiting, the passenger's destination, a value indicating if the passenger is on a bus, and the total time the passenger has waited.
- It includes methods for the passenger to board and disembark the bus, as well as to increment the wait time if the passenger is not on a bus.

Route (route.py)

- The route entity contains attributes including the name of the route and length.
- It contains an array of the roads contained, of the traffic lights on the route, of the bus stops, the stop signs, of the crosswalks, and an array of all the entities on the route.
- There includes methods to add entities to the roads and bus stops as well as to get the location of the bus and to get the next location in the route
- There includes a method to sort the entities where the entities in the entities array are sorted by their position.

Road Segment (roadSegment.py)

- Segment ID defines the unique road that the bus is on
- Other attributes include the speed limit of the road and the start and end position of the road segment on the route
- The congestion level sees the traffic on the road, where no traffic indicates a congestion level of 1 (no congestion), to a maximum of 2.3 (determined by the congestion index regression function)

Stop Sign (stopSign.py)

- Position stores the location of the stop sign
- ID is the unique identifier for the stop sign
- Scaling factor is used for passenger generation, has a default value of 1.0

Traffic Light (trafficLight.py)

- Light ID is a unique identifier for each traffic light instance
- Position describes the actual location of the traffic light
- Initial State of traffic light represents the starting state of the light when this instance is created, which is set to 'red'
 - Switches between 2 options, 'red' and 'green'
- Cycle Length is a dictionary that defines the duration of each state of the traffic light

- `update_light`: decrements the timer and change the traffic light's state once the timer reaches zero. The light alternates between 'red' and 'green', resetting the timer according to the cycle length for the new state
- `is_green`: A simple method that checks if the current state of the traffic light is 'green'.

Simulation Driver (`simulationDriver.py`)

- Final driver file for running the simulation which oversees setting up all the traffic entities and running the simulation
- `route.Route`: Represents a bus route with the name "gold".
- `roadSegment.RoadSegment`: Represents segments of the road with attributes (ID, speed limit, lanes, start position, and end position)
- `BusStop.BusStop(stop_id, position, route)`: Represents bus stops with a unique identifier, location of the stop, and associated route.
- `stopSign.StopSign`: Represents stop signs with a unique identifier and position.
- `trafficLight.TrafficLight`: Represents traffic lights with a unique identifier and position.
- `CrossWalk.CrossWalk`: Represents crosswalks with a unique identifier and position.
- `bus.Bus`: Represents buses with a unique identifier, route it's on, capacity, and an optional position attribute.
- `SimulationExecutive.SimulationExecutive`: class responsible for running the simulation, initialized with a start time, end time, a route, and a list of buses.

The simulation process starts at 7:00 AM and ends at 8:00 PM. During this time, the script initializes the route and its associated entities like roads, bus stops, stop signs, traffic lights, and crosswalks. Buses are also created and added to the simulation. The **SimulationExecutive** is then tasked with managing the simulation, likely by updating the states of all entities (like moving buses, changing traffic light states) at each timestep from the start to the end time.

Simulation Executive (`SimulationExecutive.py`)

- `time`: current simulation time.
- `end_time`: time at which the simulation ends.
- `buses`: list of bus entities participating in the simulation.
- `route`: route entity that buses follow, containing road segments, bus stops, traffic lights, stop signs, and crosswalks.
- `Fig, ax`: Matplotlib figure and axis for plotting the simulation.

- `bus_plots`, `entity_plots`: lists for storing plot objects relating to buses and other entities.
- `stop_timestep_counter`: counter to manage the update frequency for bus stops.
- `total_passengers`: total number of passengers that have been waiting at stops.
- `total_wait_time`: accumulated wait time for all the passengers.
- `__init__`: Initializes the simulation with start time, end time, route, and buses. It also sets up plotting utilities for visualization.
- `update_simulation_state`: updates the state of the simulation at each timestep. This includes updating the status and position of each bus, updating the number of passengers and wait time at each bus stop, and updating the state of traffic lights.
- `plot_simulation_init`: Sets up the initial state of the plot that will visualize the simulation.
- `plot_simulation_update`: Updates visualization of the simulation at each frame of the animation. It represents buses, bus stops, and other traffic entities graphically on the plot.
- `run`: Executes the simulation. If `animation` is `True`, it uses Matplotlib's `FuncAnimation` to animate the simulation; otherwise, it simply updates the simulation state until the `end_time` is reached. It prints the total and average wait time for passengers at the end of the simulation.

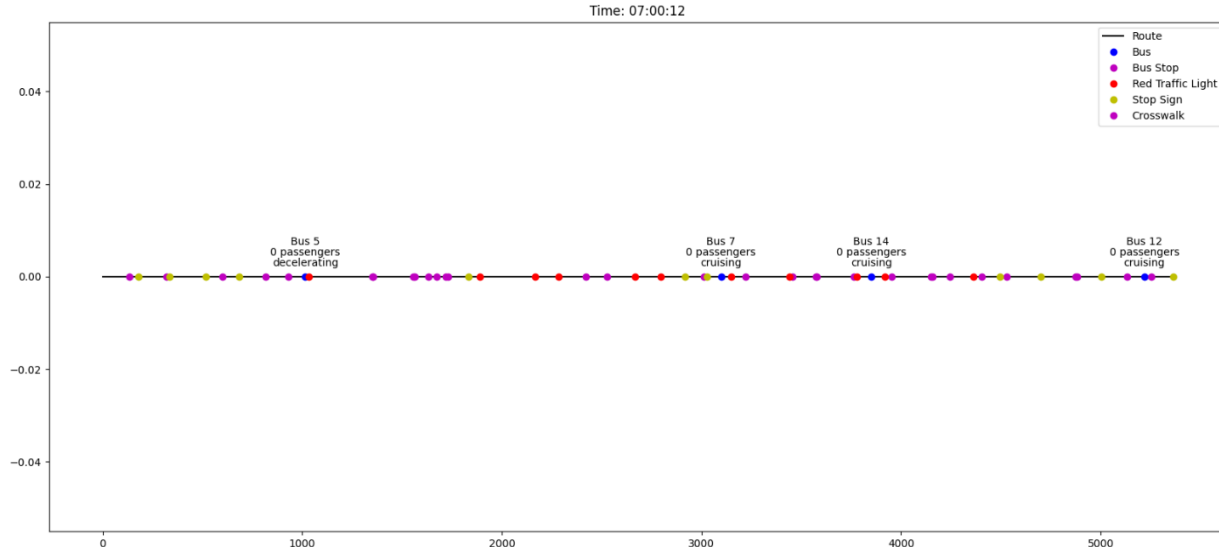
Random Event (`randomEvent.py`)

- The random event class implements logic to generate random data for the passengers on the bus route.
- The drop off random method randomly selects a destination bus stop for a passenger who is boarding at the stop. It makes sure it is not the same stop as the pickup. The randomness is implemented to model the unpredictable nature of passengers and their destinations.
- A method, `passenger distribution`, is implemented which models the distribution of passenger arrivals at stops, with peak times around 10:00AM and 4:00PM. The function incorporates the sum of two normal distributions to reflect the increased likelihood of passengers arriving during peak hours. The scaling factor is incorporated to adjust the intensity of the distribution and allowing for simulation scalability.
- A stop sign wait time method determines a random wait time at a stop sign and is impacted by the time of day. This models the fluctuation in stop durations based on traffic conditions. The scaling factor is incorporated to adjust the range of possible wait times and simulate daily traffic patterns.
- A crosswalk wait time method is implemented, like the stop sign wait time method, which calculates a random wait time at a crosswalk. It does not use the scaling factor parameter but incorporates a distribution function that considers the time of the day.

Validation and Experiments

To collect real bus data to develop and validate our model, we requested and received access to the GT Bus API via Transloc (a shuttle software company that records various GT bus information in real time). This API gives access to bus location, occupation, and route information at any given point in time. As most buses run from the time frame of 7 AM to 7 PM, we wrote a python script to collect the location and occupation information for all buses in that time frame. This script essentially queried the API every 5 seconds and saved the bus information as timestamped JSONs. We can then later use this data to set certain constants in our model to reflect GT Buses such as speed, occupancy information, number of buses, and more. In addition to data collected every 5 seconds for 12 hours, we also collected data every second for 1 hour for more finely grained data.

We first used our data to model the blue route of the GT bus system. The first experiment we conducted was by placing the starting positions at starting positions based off the data that we collected. We set the time frame to be from 7 AM to 7 PM. A snapshot of the animation for the model can be seen below.



The line represents the 1d representation of the blue route and the various colored dots represent different things along the route such as stop signs and bus stops that affect the buses' movement. The animation also shows various information about the buses on the route such as their status and passenger information. Viewing the animation of the model allowed us to see the functionality of the model. This included both deceleration/acceleration for buses when stopping due to red traffic lights, stop signs, bus stops, and crosswalks and accelerating

afterwards. Furthermore, we were able to see the traffic lights along the route change according to the passed in schedule.

We also implemented code to automatically calculate the average wait time after running the simulation for a given time frame. This wait time was calculated by taking the average of the wait time for the randomly generated passengers at stops. For experimentation, we tried running the model between 10 AM to 11 AM (higher traffic hours) vs 1-2 PM (lower traffic hours) and comparing the average wait times. The prediction was that the 1-2 PM time frame would have lower average wait times as the roads would be less congested, and buses would thus be able to travel their routes faster. This hypothesis was confirmed as the higher traffic hours had an average wait time of 30 minutes and 41 seconds while the lower traffic hours had a lower average wait time of 26 minutes.

To validate our model, we used the more finely grained data to calculate the differences between our model's predicted arrival times, the Transloc predictions, and the actual bus arrival times. This was done by first setting time frames of 10, 20, and 30 minutes. The start time window was set at 11:30 AM. The error was calculated by taking the absolute value of the difference between the predicted/estimated bus arrival time vs the actual time that the bus arrived at these stops. The mean of these errors for all buses along all stops within the window were then calculated to get the average time error for the model. The results can be seen below.

```
After 10 minutes
Average time error for model (min): 2.332430555555553
Average time error for Transloc (min): 1.965625
After 20 minutes
Average time error for model (min): 2.62078125
Average time error for Transloc (min): 2.238263888888889
After 30 minutes
Average time error for model (min): 3.1134118716931214
Average time error for Transloc (min): 2.8209126984126986
```

As we can see, our model was unable to beat the estimated times provided by Transloc. However, the difference is relatively small (less than half a minute). Furthermore, it appears that for both our model and the Transloc model, the longer the time frame, the larger the error. This makes sense as the longer the time frame, the more time the model has to accumulate error. Furthermore, once the model is either ahead or behind of the actual bus route, it might continue falling more and more ahead/behind.

Optimal Bus Starting Points

After validating our simulation, we can use our simulation to perform analysis on the bus route to make decisions to improve the efficiency of the route. We decided to solve the problem of deciding what the optimal starting points for buses on the bus route are. This is important because an optimal bus system would have buses that are separated from each other. However, a common occurrence of bus systems is for the buses to bunch together. This is suboptimal because it increases the wait times for passengers on the bus route. Although finding optimal starting points for buses along the route will not solve bus bunching in its entirety, it would help delay the time it takes for buses to become bunched together.

Bus bunching is common because buses that are behind schedule tend to fall further behind and buses ahead of schedule tend to get further ahead. When a bus falls behind schedule, it spends more time at stops picking up passengers, causing it to fall even further behind. Meanwhile, the bus ahead, with fewer passengers to pick up, can maintain or even extend its lead.

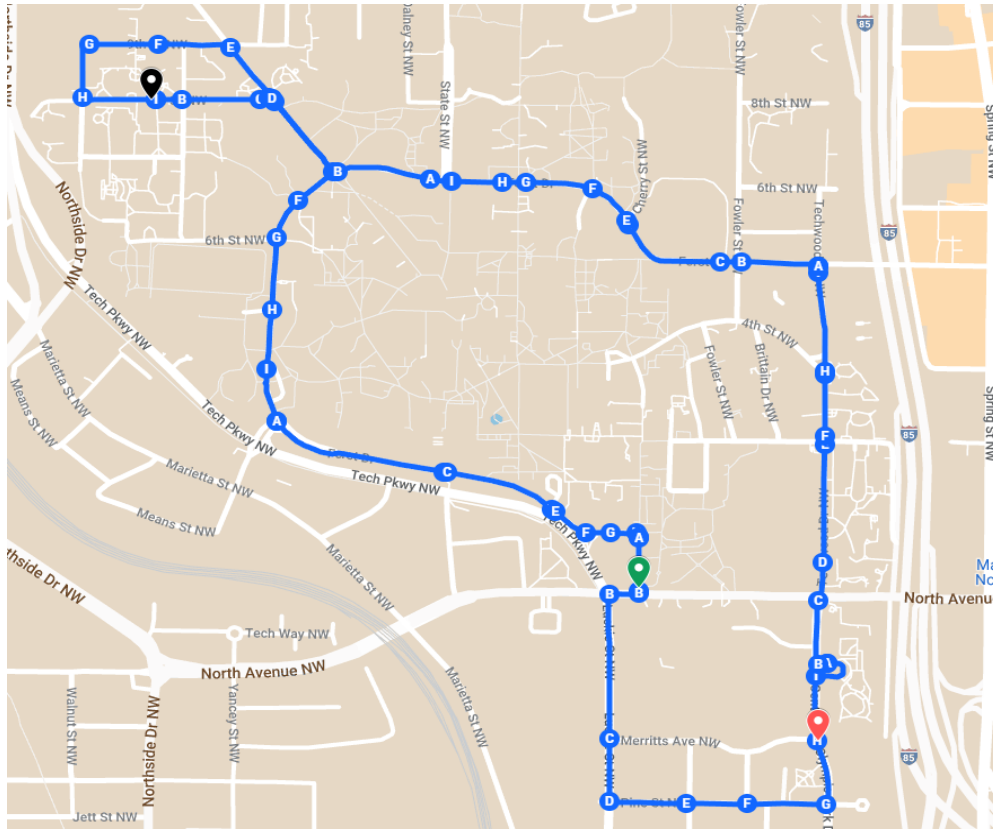
The optimal starting points for the buses would be setting waypoints along the route such that each waypoint is the same time distance from each other, meaning it takes the same time to drive a bus from one waypoint to the next. A simple approach would be to set waypoints such that the spatial distance along the route is the same between consecutive waypoints. However, buses are slowed down by bus stops, traffic lights, stop signs, and crosswalks. This wouldn't be factored into that simple approach.

Another simple approach, using our simulation, would be to run one bus around the route for several iterations of the route. We would need several iterations because some of our calculations are not deterministic. By averaging several loops of the bus, we can find expected values for our route. If we had k buses to place on the route, we could break up one loop that the bus performs into k segments that take the bus $1/k$ th of the total driving time of the loop. This would ensure our waypoints are set at equidistant points based on time and not spatial distance. However, this approach does not factor in the impact of one bus on all other buses within the bus route. If we add a bus to the route, it will pick up passengers along the route and drop them off at their destination. Any people picked up by this bus will not have to be picked up by other buses on the bus route. This is not accounted for when we run a single bus through our simulation to determine the optimal starting locations for buses.

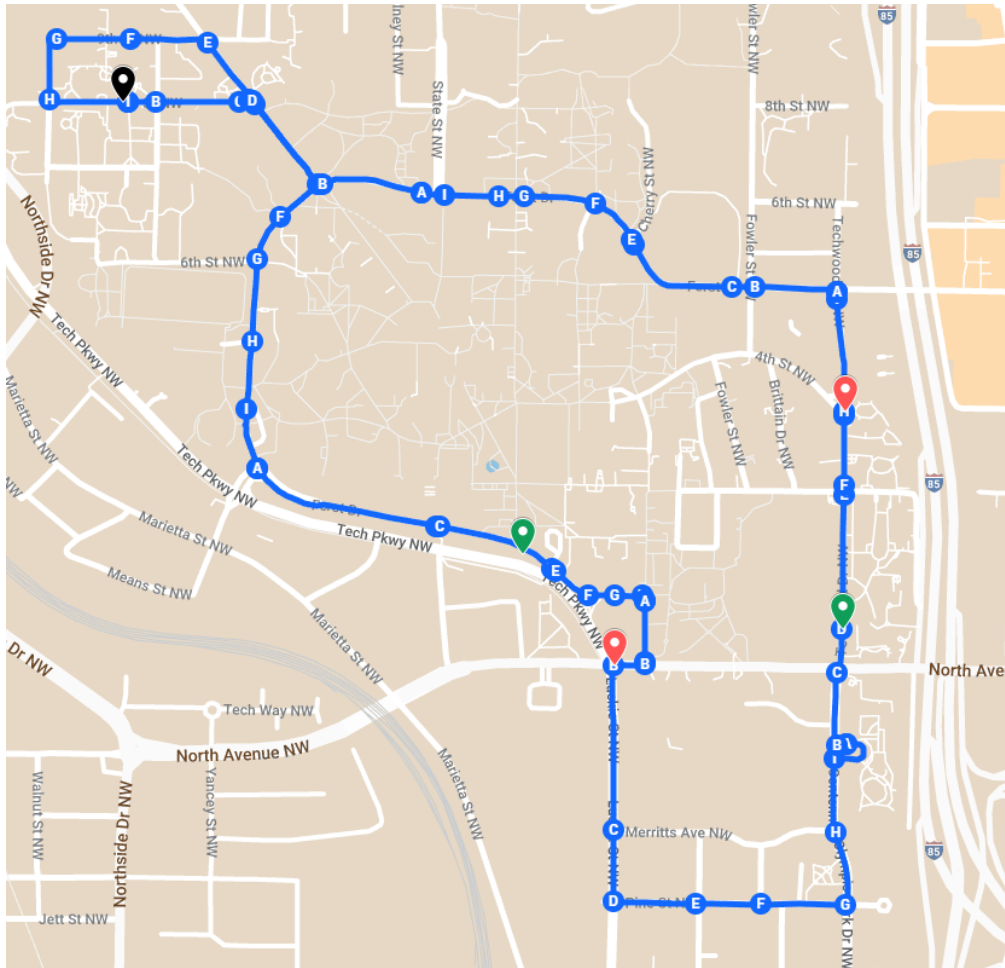
The approach we decided to utilize was to start k buses at k points that are spatially equidistant from each other along the bus route. We would run the simulation n times and determine which two buses were the first to bunch together. We would then take a step of size η for both buses that were bunched. The bus that caught up to the other bus would be moved η meters to the left and the bus that was caught would be moved η meters to the right. This approach is very similar to gradient descent where small steps are taken to minimize an objective function. In our situation, we are trying to maximize the time it takes for any two buses in the bus system to bunch together or equivalently minimize the negative of the time it takes for buses to bunch. After performing n iterations of steps to optimize our objective, we can use the new starting waypoints of the buses as the optimal places to start buses along the bus route at the beginning of the day. Because of non-determinism in our simulation, we performed b simulations for every iteration and took the majority vote to determine which bus was most likely to bunch for a given initial configuration.

Because of the fast speed of iterating our simulation, we chose a very small step size $\eta = 5$ meter for our optimization. We chose n , the number of optimization iterations, using early stopping with a patience of 3 iterations, meaning that we ran our simulation until the expected time for bus bunching didn't improve for 3 iterations in a row. We performed our experiment for k , the number of buses, equal to 2 and 3.

Below are images that show 3 types of markers. The black marker is a bus stop that we always fix and never changes during the iteration. This ensures the relative position of the other markers is more comparable to each other. The green marker is a way point determined using our simulation, and the red marker is the initial way point used as the initial values to our optimization problem. The first one is for $k = 2$.



The next map is the results when setting $k = 3$.



Conclusion

Overall, we were able to create a relatively accurate 1D visualization of buses along a route. Although we specifically created the blue route for our testing and experiments, the underlying model is route agnostic and new routes can be easily created by using a combination of roads and stops to represent the bus's path. Furthermore, our project was able to leverage multiple pieces of information such as traffic light states, positions of other buses, and pickup times to show real life phenomena, such as bus bunching. While our models' predictions were slightly worse than Transloc's predicted times, we have still made a much richer and more comprehensible model than Transloc's simple predicted arrival times.

However, our project also has limitations and areas for future improvement. One limitation is that our model does not account for the impact of other bus routes, as passengers may choose alternative routes. Additionally, our model currently does not incorporate the effects of driving around curves or turns in the route, which can affect bus speeds.

Future improvements to this model could enhance its accuracy to real world behavior. One area of improvement could involve more detailed passenger behavior. The current model uses a normal distribution to model the rate of waiting passengers arriving at bus stops, but a more accurate distribution could account for surges during specific times, like before and after class periods or during peak hours. Bus stop popularity could also be considered. The current model uses the same distribution for the rate of waiting passengers arriving for every stop and chooses randomly with equal probability which stop a passenger will exit the bus. A more accurate model would have a higher number of waiting passengers and exiting passengers at the most popular stops.

This project could also be extended to model the other routes in the GT bus system. This could allow for the optimization of the transportation system as a whole and reveal any differences in behavior between the routes.

References

- [1] Y. Lin, X. Yang, N. Zou, and L. Jia, "Real-Time Bus Arrival Time Prediction: Case Study for Jinan, China," *Journal of Transportation Engineering*, vol. 139, no. 11, pp. 1133–1140, Nov. 2013, doi: [https://doi.org/10.1061/\(asce\)te.1943-5436.0000589](https://doi.org/10.1061/(asce)te.1943-5436.0000589).
- [2] R. Jeong and L. R. Rilett, "Prediction Model of Bus Arrival Time for Real-Time Applications," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 1927, no. 1, pp. 195–204, Jan. 2005, doi: <https://doi.org/10.1177/0361198105192700123>.
- [3] B. Yu, W. H. K. Lam, and M. L. Tam, "Bus arrival time prediction at bus stop with multiple routes," *Transportation Research Part C: Emerging Technologies*, vol. 19, no. 6, pp. 1157–1170, Dec. 2011, doi: <https://doi.org/10.1016/j.trc.2011.01.003>.
- [4] A. Shalaby and A. Farhan, "Prediction Model of Bus Arrival and Departure Times Using AVL and APC Data," *Journal of Public Transportation*, vol. 7, no. 1, pp. 41–61, 2004.
- [5] M. Chen, X. Liu, J. Xia, and S. I. Chien, "A Dynamic Bus-Arrival Time Prediction Model Based on APC Data," *Computer-Aided Civil and Infrastructure Engineering*, vol. 19, no. 5, pp. 364–376, Sep. 2004, doi: <https://doi.org/10.1111/j.1467-8667.2004.00363.x>.
- [6] Y. Pei, S. Biswas, D. S. Fussell, and K. Pingali, "An elementary introduction to Kalman filtering," *Communications of the ACM*, vol. 62, no. 11, pp. 122–133, Oct. 2019, doi: <https://doi.org/10.1145/3363294>.
- [7] C. Bai, Z.-R. Peng, Q.-C. Lu, and J. Sun, "Dynamic Bus Travel Time Prediction Models on Road with Multiple Bus Routes," *Computational Intelligence and Neuroscience*, vol. 2015, pp. 1–9, 2015, doi: <https://doi.org/10.1155/2015/432389>.
- [8] B. Yu, Z.-Z. Yang, K. Chen, and B. Yu, "Hybrid model for prediction of bus arrival times at next station," *Journal of Advanced Transportation*, vol. 44, no. 3, pp. 193–204, Jul. 2010, doi: <https://doi.org/10.1002/atr.136>.
- [9] A. Achar, D. Bharathi, B. A. Kumar, and L. Vanajakshi, "Bus Arrival Time Prediction: A Spatial Kalman Filter Approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1298–1307, Mar. 2020, doi: <https://doi.org/10.1109/tits.2019.2909314>.
- [10] Z. Wall, D. Dailey, and D. Washington, "Title: An Algorithm for Predicting the Arrrival Time of Mass Transit Vehicles Using Automatic Vehicle Location Data An Algorithm for Predicting the Arrival Time of Mass Transit Vehicles Using Automatic Vehicle Location Data," 1999. Accessed: Mar. 04, 2024. [Online]. Available:

<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=1529b6a90d73f8c2ea4344c6600f079d536244e6>

[11] Shalaby, Amer & Farhan, Ali. 2004. Prediction Model of Bus Arrival and Departure Times Using AVL and APC Data. *Journal of Public Transportation*, 7 (1): 41-61.

DOI: <http://doi.org/10.5038/2375-0901.7.1.3>

[12] Comi A, Polimeni A. Bus Travel Time: Experimental Evidence and Forecasting. *Forecasting*. 2020; 2(3):309-322. <https://doi.org/10.3390/forecast2030017>

[13] Mete, S., Çelik, E., & Gül, M. (2022). Predicting the Time of Bus Arrival for Public Transportation by Time Series Models. *Journal of Transportation and Logistics*, 7(2), 441-455. <https://doi.org/10.26650/JTL.2022.953913>

[14] Steven Kornfeld, Wei Ma, and Andrew Resnikoff, "Optimizing Bus Schedules to Minimize Waiting Time", 21-393 - *Operations Research*

II. <https://www.math.cmu.edu/~af1p/Teaching/OR2/Projects/P46/G.pdf>

[15] M. V. Koppiseti, V. Kavitha and U. Ayesta, "Bus schedule for optimal bus bunching and waiting times," 2018 10th International Conference on Communication Systems & Networks (COMSNETS), Bengaluru, India, 2018, pp. 607-612, <https://doi.org/10.1109/COMSNETS.2018.8328282>

[16] G. Carra, P. Micucci, and M. Rinchuso, "Hybrid Algorithm for the Estimation of Bus Travel Times," *PLOS ONE*, vol. 15, no. 5, May 2020, <https://doi.org/10.1371/journal.pone.0232799>

[17] Noor, R. M., Rasyidi, N. B. G., Nandy, T., & Kolandaisamy, R. (2020, December 29). Campus Shuttle Bus Route Optimization Using Machine Learning Predictive Analysis: A Case Study. *Sustainability*. <https://doi.org/10.3390/su13010225>

[18] D. Panovski and T. Zaharia, "Real-Time Public Transportation Prediction with Machine Learning Algorithms," 2020 IEEE International Conference on Consumer Electronics (ICCE), Las Vegas, NV, USA, 2020, pp. 1-4, doi: 10.1109/ICCE46568.2020.9043077. keywords: {Support vector machines;Deep learning;Machine learning algorithms;Urban areas;Neural networks;Linear regression;Prediction algorithms}

[19] Shalaby, A., & Farhan, A. (2004). Prediction model of bus arrival and departure times using AVL and APCdata. *Journal of Public Transportation*, 7(1), 41–61. <https://doi.org/10.5038/2375-0901.7.1.3>

Appendix (Contributions)

- All
 - Equally contributed to the Literature Review and progress checkpoints.
- Kyle Peters:
 - Created the 1d representation of the blue bus route.
 - Performed the bus bunching analysis to determine optimal bus starting locations.
- Grace Driskill:
 - Helped with creating the conceptual model and implementing the simulation
- Zi'an Dong
 - Implemented the basic conceptual model logic
 - Created the animation for simulation model
- Anthony Ahn:
 - Wrote script to collect GT Bus API data and preprocessed the data
 - Performed experiments to show the blue bus routes using the collected positional data
 - Validated the model results by comparing with actual bus position data and Transloc estimated times
- Anushka Chaudhari
 - Created the entity classes and helped implement simulation including bus bunching and distance threshold
- Neelkanth Gandediwal
 - Created the base entity classes and implemented advanced logic in model (dynamic bus movement, distance threshold, passenger movement, etc.)